Elektronika MK-56

Электроника МК-56

Technical Reference & Microarchitecture Manual

Formal Hardware Specification, RPN Stack, Instruction Set, and Flow Control

Normative Reference for Retrocomputing Software Engineering:
Compilers, Assemblers, Disassemblers, Emulators, and Virtual Machines

Computing Family:	USSR B3-34 / MK-54 / MK-56 / MK-61
CPU Architecture:	K145 PMOS LSI Integrated Circuit Series
Computational Model:	Reverse Polish Notation (*ПОЛИЗ*)
Memory Capacity:	98 program steps, 14 floating-point registers
Publication Date:	October 2026

Based on the official USSR factory technical instruction manual (*Инструкция по эксплуатации*)
Technical translation, microarchitectural analysis, and typesetting in LaTeX.

Contents

1 Architecture and Stack **2**
1.1 Historical Context and Core Circuitry . . . 2
1.2 Numeric Format and Vacuum Fluorescent Display (VFD) . . . 2
1.3 RPN Stack Architecture (*ПОЛИЗ*) . . . 3
1.3.1 Stack Register Definitions . . . 3
1.4 Stack State Dynamics and Transition Matrices . . . 3
1.4.1 A. Data Entry and Duplication: `В↑` (*Ввод в стек* / Enter / Push) . . . 3
1.4.2 B. Operand Swap: ↔ (*Обмен X и Y* / Swap) . . . 4
1.4.3 C. Downward Circular Rotation: `F` ↻ (*Кольцевое перемещение* / Roll Down) . . . 4
1.4.4 D. Clear Accumulator: `Cx` (*Сброс X* / Clear X) . . . 4
1.4.5 E. Dyadic Arithmetic Reduction (+, −, ×, ÷) . . . 4
1.4.6 F. Hardware Subroutine Return Stack (*Стек возврата*) . . . 5

2 Memory Map and Registers **6**
2.1 Program Memory (*ОЗУ команд*) . . . 6
2.1.1 Operating Modes . . . 6
2.2 Addressable Data Registers (*Адресуемые регистры*) . . . 6
2.3 Memory Addressing Modes . . . 7

3 Instruction Set Architecture (Opcodes) **8**
3.1 Pseudo-Hexadecimal VFD Display Conventions . . . 8
3.2 Official Instruction Set Table (Таблица 4) . . . 8

4 Control Flow and Branching **11**
4.1 Direct Unconditional Jump: `БП` (*Безусловный переход*) . . . 11
4.2 Subroutine Call and Return: `ПП` and `В/О` . . . 11
4.3 Inverted Conditional Branch Mechanics . . . 11
4.3.1 Formal Conditional Branch Specifications . . . 12
4.3.2 Compiler Code Generation Pattern for Conditional Constructs . . . 12
4.4 Hardware Loop Counters: `F L0`, `F L1`, `F L2`, `F L3` . . . 12
4.5 Indirect Branching and Dynamic Dispatch . . . 13

5 Architectural Specification Summary for Compiler Engineers **14**
5.1 Practical IR Code Generation Guidelines . . . 14

1 Architecture and Stack

1.1 Historical Context and Core Circuitry

The **Elektronika MK-56** (*Электроника МК-56*) is the desktop mains-powered variant (220 V AC, 50 Hz) of the second-generation programmable calculator family manufactured in the USSR. It shares microarchitecture, binary instruction set encoding, execution timing, and cycle dynamics with the handheld models **Elektronika B3-34** (*Электроника Б3-34*) and **Elektronika MK-54** (*Электроника МК-54*).

The computational core is constructed around high-density PMOS LSI integrated circuits from the **K145 series** (*серия К145*):

- **D4 (*К145ИК1302* / K145IK1302):** Main control microcomputer, sequencer, and keyboard decoder. Manages instruction fetch-decode-execute cycles, keypad scanning, VFD refresh routines, and program counter (PC) addressing.
- **D5 (*К145ИК1303* / K145IK1303):** Arithmetic and transcendent function coprocessor. Implements CORDIC micro-algorithms and series expansions for trigonometric, logarithmic, and exponential operations.
- **D6 and D7 (*К145ИР2* / K145IR2):** Dynamic 1008-bit serial shift-register delay lines providing circular storage for the 14 addressable data registers, the 4 operational stack levels, and the subroutine return stack.
- **D8 (*К145АП1* / K145AP1):** Multi-phase clock generator and dynamic timing pulse driver for the PMOS circuitry.
- **D1, D2, D3 (*К161ПП1* / *К161КН1* / K161 series):** High-voltage driver ICs switching phosphor anodes and grid control lines for the vacuum fluorescent display.

1.2 Numeric Format and Vacuum Fluorescent Display (VFD)

The calculator is equipped with a green-teal 12-digit vacuum fluorescent display (VFD, in Russian: *вакуумно-люминесцентный индикатор*):

- **Mantissa:** 8 decimal significant digits plus 1 sign position.
- **Exponent:** 2 decimal digits plus 1 exponent sign position.
- **Dynamic Representation Range:**

$$\pm 1 \times 10^{-99} \leq |x| \leq 9.9999999 \times 10^{99}$$

- **Fixed-Point vs. Scientific Display Modes:**
  - Values within $[10^{-7}, 10^{8} - 1]$ automatically display in standard fixed-point decimal notation with the decimal point located at the corresponding place value.
  - Values outside this range, or upon manual entry via the ВП key (*Ввод Порядка* – Enter Exponent), transition to scientific floating-point notation:

$$\pm M.MMMMMMM \quad \pm EE$$

- **Error Conditions and Overflow:** Arithmetic singularities (division by zero, square root of negative values, non-positive logarithms) or exponent overflow ($> +99$) cause the sequencer to halt automatic execution and display ЕГГОГ (a 7-segment representation of *ERROR* using Soviet segment glyphs).

1.3 RPN Stack Architecture (*ПОЛИЗ*)

The processor executes expressions using Reverse Polish Notation (in Soviet computer science: *ПОЛИЗ* – *Польская Инверсная Запись* / Polish Inverted Notation). Evaluation requires no deferred execution parentheses; instead, it utilizes a **4-level hardware operational stack** (X, Y, Z, T) coupled with an automatic **shadow backup register** (X_1):

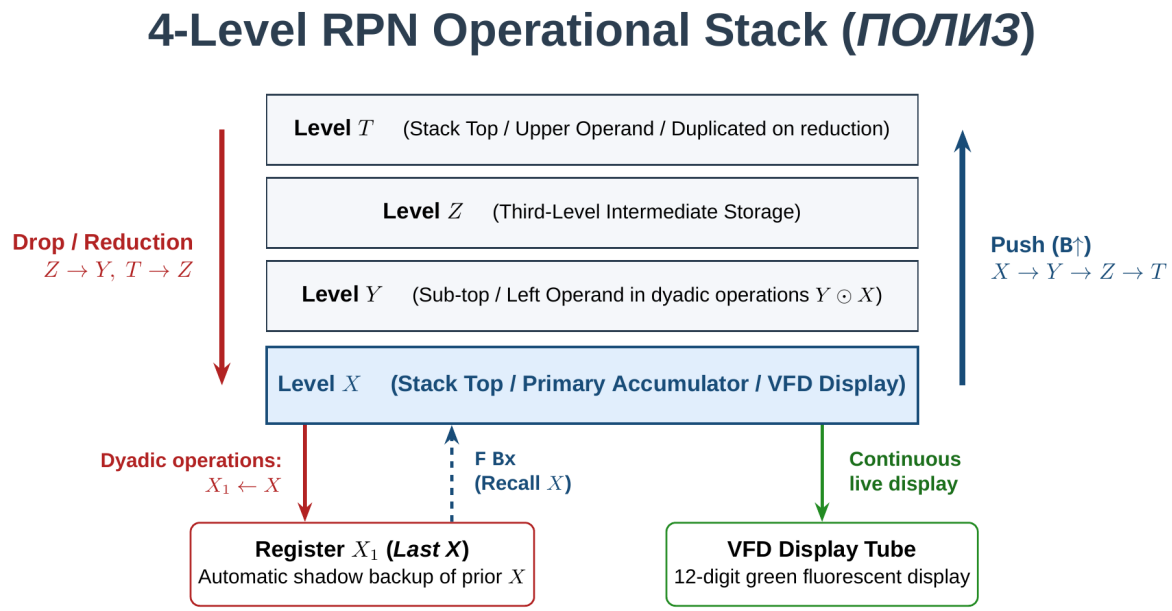


Figure 1: 4-Level RPN Operational Stack (*ПОЛИЗ*) Architecture, shadow register X_1, and VFD display.

1.3.1 Stack Register Definitions

1. **Register X (*Регистр X*):** Stack top (*Вершина стека*). This is the only register continuously reflected on the VFD. It serves as the primary accumulator for all monadic operations and the target for dyadic results.
2. **Register Y (*Регистр Y*):** Sub-top. Holds the left operand (first operand) in binary operations: $Y \odot X$.
3. **Register Z (*Регистр Z*):** Third-level intermediate storage.
4. **Register T (*Регистр T*):** Fourth level of the stack.
5. **Register X_1 (*Регистр X_1 / Предыдущий результат*):** Shadow backup register (*Last X*). Automatically captures the previous contents of X immediately prior to executing any binary arithmetic operation or transcendent function. Restorable into X using the key sequence F Bx (*Восстановление X*).

1.4 Stack State Dynamics and Transition Matrices

1.4.1 A. Data Entry and Duplication: B↑ (*Ввод в стек* / Enter / Push)

Pressing B↑ pushes all stack levels upward:

$$\begin{pmatrix} X \\ Y \\ Z \\ T \end{pmatrix} \xrightarrow{\text{B}\uparrow} \begin{pmatrix} X \\ X \\ Y \\ Z \end{pmatrix}$$

The value in X is copied to Y. The previous value of T is dropped. The register X is preserved, ready to receive incoming digits without overwriting Y.

1.4.2 B. Operand Swap: $\leftrightarrow$ (*Обмен X и Y* / Swap)

Exchanges the contents of the stack top (X) and sub-top (Y) without altering Z and T:

$$\begin{pmatrix} X \\ Y \\ Z \\ T \end{pmatrix} \xrightarrow{\leftrightarrow} \begin{pmatrix} Y \\ X \\ Z \\ T \end{pmatrix}$$

1.4.3 C. Downward Circular Rotation: F ↻ (*Кольцевое перемещение* / Roll Down)

Cyclically rotates the four stack levels downward:

$$\begin{pmatrix} X \\ Y \\ Z \\ T \end{pmatrix} \xrightarrow{\text{F}\,\circlearrowleft} \begin{pmatrix} Y \\ Z \\ T \\ X \end{pmatrix}$$

Performing four consecutive rotations (F ↻ $\times 4$) restores the stack precisely to its initial state.

1.4.4 D. Clear Accumulator: Cx (*Сброс X* / Clear X)

Clears the active mantissa in X to zero without modifying depth or data in Y, Z, and T:

$$\begin{pmatrix} X \\ Y \\ Z \\ T \end{pmatrix} \xrightarrow{\text{Cx}} \begin{pmatrix} 0 \\ Y \\ Z \\ T \end{pmatrix}$$

1.4.5 E. Dyadic Arithmetic Reduction ($+$, $-$, $\times$, $\div$)

In binary operations where Y is the left-hand operand and X is the right-hand operand ($Y \odot X$):

1. The right operand X is stored into the shadow backup register X_1 ($X_1 \leftarrow X$).
2. The evaluated result $Y \odot X$ is written to X.
3. Level Z drops to Y ($Y \leftarrow Z$).
4. Level T drops to Z ($Z \leftarrow T$).
5. Level T **duplicates its value** ($T \leftarrow T$), a distinctive characteristic of PMOS RPN hardware architectures.

$$\begin{pmatrix} X \\ Y \\ Z \\ T \end{pmatrix} \xrightarrow{+,\,-,\,\times,\,\div} \begin{pmatrix} Y \odot X \\ Z \\ T \\ T \end{pmatrix}, \qquad X_1 \leftarrow X$$

1.4.6 F. Hardware Subroutine Return Stack (*Стек возврата*)

The K145IK1302 core integrates a **dedicated 5-level LIFO hardware return stack**:

- Each subroutine call (ПП) pushes the next sequential address ($PC + 2$) onto the return stack.
- Each subroutine return (В/О) pops the top address from the return stack and branches to it.
- **Nesting limit:** Nesting more than 5 levels causes return stack overflow (*circular overwrite*), dropping the oldest return address.

2 Memory Map and Registers

2.1 Program Memory (*ОЗУ команд*)

- **Total Capacity:** 98 program steps.
- **Address Space:** 2-digit linear decimal addressing from `00` to `97`.
- **Instruction Format:** 1 byte per step. Branch instructions taking immediate addresses span two contiguous steps:
  - Step N: Branch opcode (e.g., `51` for unconditional jump `БП`).
  - Step $N+1$: 2-digit decimal target address (e.g., `25`).

2.1.1 Operating Modes

1. **Automatic / Run Mode (*АВТ* / `F АВТ` or *Р* switch):** Executes program steps sequentially at full clock speed upon pressing `С/П` (*Старт/Пуск*).
2. **Programming / Edit Mode (*ПРГ* / `F ПРГ` or *ПРГ* switch):** Allows direct reading and writing of program RAM. The VFD adopts the canonical format:

   AA CC

   Where **AA** represents the active program address (`00` to `97`) and **CC** is the opcode byte stored at that address. The keys `ШГ` $\rightarrow$ and `ШГ` $\leftarrow$ step forward and backward through memory.

2.2 Addressable Data Registers (*Адресуемые регистры*)

The machine features **14 non-volatile static/dynamic data registers** while powered:

$$RG0, RG1, RG2, RG3, RG4, RG5, RG6, RG7, RG8, RG9, RGa, RGb, RGc, RGd$$

Each register stores a full floating-point decimal word (8-digit mantissa and 2-digit signed exponent).

Register	Key	Behavior in Indirect Mode (`К`)	Common Architectural Use
$RG0$	`0`	Auto-decrement by 1 ($RG0 \leftarrow RG0 - 1$)	Loop counter / Downward pointer
$RG1$	`1`	Auto-decrement by 1 ($RG1 \leftarrow RG1 - 1$)	Loop counter / Downward pointer
$RG2$	`2`	Auto-decrement by 1 ($RG2 \leftarrow RG2 - 1$)	Loop counter / Downward pointer
$RG3$	`3`	Auto-decrement by 1 ($RG3 \leftarrow RG3 - 1$)	Loop counter / Downward pointer
$RG4$	`4`	Auto-increment by 1 ($RG4 \leftarrow RG4 + 1$)	Upward array/memory pointer
$RG5$	`5`	Auto-increment by 1 ($RG5 \leftarrow RG5 + 1$)	Upward array/memory pointer
$RG6$	`6`	Auto-increment by 1 ($RG6 \leftarrow RG6 + 1$)	Upward array/memory pointer
$RG7$	`7`	No auto-modification ($RG7 \leftarrow RG7$)	General-purpose data register
$RG8$	`8`	No auto-modification ($RG8 \leftarrow RG8$)	General-purpose data register
$RG9$	`9`	No auto-modification ($RG9 \leftarrow RG9$)	General-purpose data register
RGa	`.` (a)	No auto-modification ($RGa \leftarrow RGa$)	General-purpose data register
RGb	`/-/` (b)	No auto-modification ($RGb \leftarrow RGb$)	General-purpose data register
RGc	`ВП` (c)	No auto-modification ($RGc \leftarrow RGc$)	General-purpose data register
RGd	`Сх` (d)	No auto-modification ($RGd \leftarrow RGd$)	General-purpose data register

2.3 Memory Addressing Modes

1. **Direct Addressing:**
   - **Store into Register:** x→П $[R]$ (*Запись X в регистр*). Copies accumulator X into register R ($0 \leq R \leq d$). Accumulator X is preserved.
   - **Recall from Register:** П→x $[R]$ (*Вызов из регистра в X*). Pushes the operational stack upward and loads register R into X.
2. **Indirect Addressing (Prefix К):**
   - **Indirect Store:** К x→П $[R]$. Uses the integer portion of register R as the effective target pointer to store X.
   - **Indirect Recall:** К П→x $[R]$. Uses the integer portion of register R as the source pointer to load data into X.

Critical Automatic Pointer Modification Rule

Prior to executing an indirect transfer or indirect branch with prefix К:

- If $R \in \{0, 1, 2, 3\}$: Pointer register R is **decremented by 1** ($R \leftarrow R - 1$).
- If $R \in \{4, 5, 6\}$: Pointer register R is **incremented by 1** ($R \leftarrow R + 1$).
- If $R \in \{7, 8, 9, a, b, c, d\}$: Pointer register R remains **unmodified**.

3 Instruction Set Architecture (Opcodes)

The following catalog provides the complete instruction set of the Elektronika MK-56 transcribed from the factory manual (**Table 4 - Таблица 4**).

3.1 Pseudo-Hexadecimal VFD Display Conventions

The K145IK1302 microcontroller encodes 4-bit nibbles (`0x0` to `0xF`) on 7-segment displays according to Soviet design standards:

Hex Nibble	VFD Glyph	Significance / Illuminated Segments
`0x0` – `0x9`	`0` – `9`	Standard decimal digits
`0xA` (10)	`-`	Middle horizontal segment (dash)
`0xB` (11)	`L`	Lower vertical and bottom horizontal segments (capital L)
`0xC` (12)	`C`	Top, upper/lower left, and bottom segments (capital C)
`0xD` (13)	`Г`	Cyrillic Gamma (top and upper left segments)
`0xE` (14)	`E`	Standard capital E
`0xF` (15)	*blank*	Digit unlit (blank space)

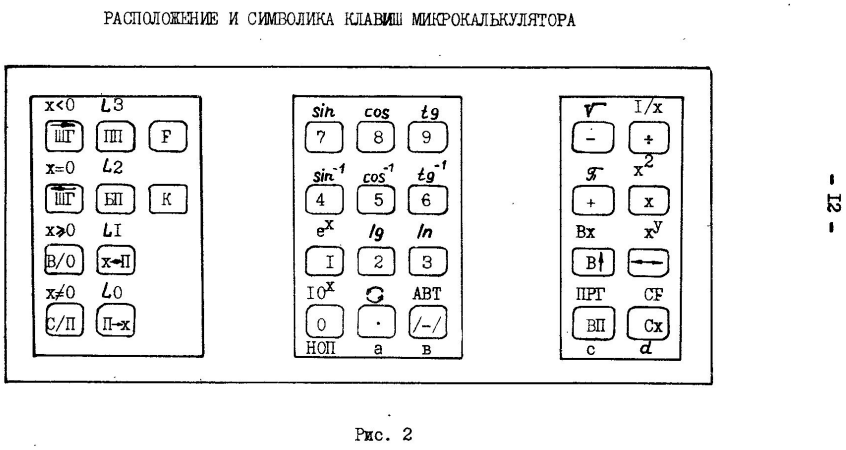


Рис. 2

- 12 -

Figure 2: Physical Keyboard Layout and Key Symbols of the Elektronika MK-56 (Figure 2 from the official USSR factory manual).

3.2 Official Instruction Set Table (Таблица 4)

Key (Cyrillic)	English Name	VFD	Hex	Formal Semantics & Stack Effect
`0`	Zero	`00`	`0x00`	Loads literal digit 0 into accumulator X

Key (Cyrillic)	English Name	VFD	Hex	Formal Semantics & Stack Effect
1	One	01	0x01	Loads literal digit 1 into accumulator X
2	Two	02	0x02	Loads literal digit 2 into accumulator X
3	Three	03	0x03	Loads literal digit 3 into accumulator X
4	Four	04	0x04	Loads literal digit 4 into accumulator X
5	Five	05	0x05	Loads literal digit 5 into accumulator X
6	Six	06	0x06	Loads literal digit 6 into accumulator X
7	Seven	07	0x07	Loads literal digit 7 into accumulator X
8	Eight	08	0x08	Loads literal digit 8 into accumulator X
9	Nine	09	0x09	Loads literal digit 9 into accumulator X
.	Decimal Point	0-	0x0A	Decimal point separator for active mantissa
/-/	Change Sign	0L	0x0B	Inverts sign of accumulator: $X \leftarrow -X$
ВП	Enter Exponent	0C	0x0C	Switches to scientific exponent entry (EE)
Cx	Clear X	0Г	0x0D	Clears accumulator ($X \leftarrow 0$); stack unchanged
В↑	Enter / Push	0E	0x0E	Push: $X \to Y, Y \to Z, Z \to T$; preserves X
+	Addition	10	0x10	$X \leftarrow Y + X;\ Z \to Y;\ T \to Z;\ X_1 \leftarrow X$
-	Subtraction	11	0x11	$X \leftarrow Y - X;\ Z \to Y;\ T \to Z;\ X_1 \leftarrow X$
×	Multiplication	12	0x12	$X \leftarrow Y \times X;\ Z \to Y;\ T \to Z;\ X_1 \leftarrow X$
÷	Division	13	0x13	$X \leftarrow Y/X;\ Z \to Y;\ T \to Z;\ X_1 \leftarrow X$ ($X \neq 0$)
↔	Swap X-Y	14	0x14	Swap: $X \leftrightarrow Y$; levels Z and T unaffected
F 10^x	Ten to the Power X	15	0x15	$X \leftarrow 10^X;\ X_1 \leftarrow X$
F e^x	Exponential	16	0x16	$X \leftarrow e^X;\ X_1 \leftarrow X$
F lg	Common Log (Base 10)	17	0x17	$X \leftarrow \log_{10}(X);\ X_1 \leftarrow X$ ($X > 0$)
F ln	Natural Log (Base e)	18	0x18	$X \leftarrow \ln(X);\ X_1 \leftarrow X$ ($X > 0$)
F arcsin	Arc Sine	19	0x19	$X \leftarrow \arcsin(X);\ X_1 \leftarrow X$ ($\lvert X\rvert \leq 1$)
F arccos	Arc Cosine	1-	0x1A	$X \leftarrow \arccos(X);\ X_1 \leftarrow X$ ($\lvert X\rvert \leq 1$)
F arctg	Arc Tangent	1L	0x1B	$X \leftarrow \arctan(X);\ X_1 \leftarrow X$
F sin	Sine	1C	0x1C	$X \leftarrow \sin(X)$; angular mode per toggle switch
F cos	Cosine	1Г	0x1D	$X \leftarrow \cos(X)$; angular mode per toggle switch
F tg	Tangent	1E	0x1E	$X \leftarrow \tan(X)$; angular mode per toggle switch
F π	Constant Pi	20	0x20	Loads $\pi \approx 3.1415926$ into X; pushes stack
F $\sqrt{x}$	Square Root	21	0x21	$X \leftarrow \sqrt{X};\ X_1 \leftarrow X$ ($X \geq 0$)
F x^2	Square	22	0x22	$X \leftarrow X^2;\ X_1 \leftarrow X$
F 1/x	Reciprocal	23	0x23	$X \leftarrow 1/X;\ X_1 \leftarrow X$ ($X \neq 0$)
F x^y	Arbitrary Power	24	0x24	$X \leftarrow Y^X;\ Z \to Y;\ T \to Z;\ X_1 \leftarrow X$
x→П 0	Store to RG0	40	0x40	$RG0 \leftarrow X$; accumulator X preserved
x→П 1	Store to RG1	41	0x41	$RG1 \leftarrow X$; accumulator X preserved
x→П 2	Store to RG2	42	0x42	$RG2 \leftarrow X$; accumulator X preserved
x→П 3	Store to RG3	43	0x43	$RG3 \leftarrow X$; accumulator X preserved
x→П 4	Store to RG4	44	0x44	$RG4 \leftarrow X$; accumulator X preserved
x→П 5	Store to RG5	45	0x45	$RG5 \leftarrow X$; accumulator X preserved
x→П 6	Store to RG6	46	0x46	$RG6 \leftarrow X$; accumulator X preserved
x→П 7	Store to RG7	47	0x47	$RG7 \leftarrow X$; accumulator X preserved
x→П 8	Store to RG8	48	0x48	$RG8 \leftarrow X$; accumulator X preserved
x→П 9	Store to RG9	49	0x49	$RG9 \leftarrow X$; accumulator X preserved
x→П a	Store to RGa	4-	0x4A	$RGa \leftarrow X$; accumulator X preserved
x→П b	Store to RGb	4L	0x4B	$RGb \leftarrow X$; accumulator X preserved
x→П c	Store to RGc	4C	0x4C	$RGc \leftarrow X$; accumulator X preserved
x→П d	Store to RGd	4Г	0x4D	$RGd \leftarrow X$; accumulator X preserved
С/П	Start / Stop	50	0x50	Halts or initiates automatic execution
БП	Unconditional Jump	51	0x51	Direct jump: $PC \leftarrow$ immediate byte $(N+1)$
В/О	Return / Reset	52	0x52	Return from subroutine (pop stack) or $PC \leftarrow$ 00
ПП	Call Subroutine	53	0x53	Push $PC + 2$ to return stack; $PC \leftarrow$ byte $(N+1)$
К НОП	No Operation (NOP)	54	0x54	No Operation; advances $PC \leftarrow PC + 1$

Key (Cyrillic)	English Name	VFD	Hex	Formal Semantics & Stack Effect
F x≠0	Branch if $X \neq 0$	57	0x57	**Inverted:** If $X = 0 \Rightarrow$ jump to $N+1$; If $X \neq 0 \Rightarrow PC+2$
F L2	Loop Counter RG2	58	0x58	$RG2 \leftarrow RG2 - 1$; If $RG2 \neq 0 \Rightarrow$ jump; If $0 \Rightarrow PC+2$
F x≥0	Branch if $X \geq 0$	59	0x59	**Inverted:** If $X < 0 \Rightarrow$ jump to $N+1$; If $X \geq 0 \Rightarrow PC+2$
F L3	Loop Counter RG3	5-	0x5A	$RG3 \leftarrow RG3 - 1$; If $RG3 \neq 0 \Rightarrow$ jump; If $0 \Rightarrow PC+2$
F L1	Loop Counter RG1	5L	0x5B	$RG1 \leftarrow RG1 - 1$; If $RG1 \neq 0 \Rightarrow$ jump; If $0 \Rightarrow PC+2$
F x<0	Branch if $X < 0$	5C	0x5C	**Inverted:** If $X \geq 0 \Rightarrow$ jump to $N+1$; If $X < 0 \Rightarrow PC+2$
F L0	Loop Counter RG0	5Г	0x5D	$RG0 \leftarrow RG0 - 1$; If $RG0 \neq 0 \Rightarrow$ jump; If $0 \Rightarrow PC+2$
F x=0	Branch if $X = 0$	5E	0x5E	**Inverted:** If $X \neq 0 \Rightarrow$ jump to $N+1$; If $X = 0 \Rightarrow PC+2$
П→x 0	Recall from RG0	60	0x60	Loads $RG0$ into X; pushes operational stack
П→x 1	Recall from RG1	61	0x61	Loads $RG1$ into X; pushes operational stack
П→x 2	Recall from RG2	62	0x62	Loads $RG2$ into X; pushes operational stack
П→x 3	Recall from RG3	63	0x63	Loads $RG3$ into X; pushes operational stack
П→x 4	Recall from RG4	64	0x64	Loads $RG4$ into X; pushes operational stack
П→x 5	Recall from RG5	65	0x65	Loads $RG5$ into X; pushes operational stack
П→x 6	Recall from RG6	66	0x66	Loads $RG6$ into X; pushes operational stack
П→x 7	Recall from RG7	67	0x67	Loads $RG7$ into X; pushes operational stack
П→x 8	Recall from RG8	68	0x68	Loads $RG8$ into X; pushes operational stack
П→x 9	Recall from RG9	69	0x69	Loads $RG9$ into X; pushes operational stack
П→x a	Recall from RGa	6-	0x6A	Loads RGa into X; pushes operational stack
П→x b	Recall from RGb	6L	0x6B	Loads RGb into X; pushes operational stack
П→x c	Recall from RGc	6C	0x6C	Loads RGc into X; pushes operational stack
П→x d	Recall from RGd	6Г	0x6D	Loads RGd into X; pushes operational stack
K x≠0 [R]	Indirect Branch $X \neq 0$	70–7Г	0x70–0x7D	Test $X \neq 0$ via pointer in $RG0..RGd$
K БП [R]	Indirect Uncond. Jump	80–8Г	0x80–0x8D	Indirect jump to target address in R
K x≥0 [R]	Indirect Branch $X \geq 0$	90–9Г	0x90–0x9D	Test $X \geq 0$ via pointer in $RG0..RGd$
K ПП [R]	Indirect Subroutine	-0–-Г	0xA0–0xAD	Indirect call to address in $RG0..RGd$
K x→П [R]	Indirect Store	L0–LГ	0xB0–0xBD	Stores X into register pointed by R
K x<0 [R]	Indirect Branch $X < 0$	C0–CГ	0xC0–0xCD	Test $X < 0$ via pointer in $RG0..RGd$
K П→x [R]	Indirect Recall	Г0–ГГ	0xD0–0xDD	Loads into X from register pointed by R
K x=0 [R]	Indirect Branch $X = 0$	E0–EГ	0xE0–0xED	Test $X = 0$ via pointer in $RG0..RGd$

4 Control Flow and Branching

4.1 Direct Unconditional Jump: БП (*Безусловный переход*)

Encoded across 2 contiguous program memory steps:

- **Step N:** Opcode `51` (БП).
- **Step $N+1$:** Target 2-digit decimal address (00 to 97).

The sequencer assigns the immediate destination to the program counter: $PC \leftarrow$ Target.

4.2 Subroutine Call and Return: ПП and В/О

- **Subroutine Call (ПП – *Переход к подпрограмме*):** 2-step instruction: Opcode `53` followed by destination address. Pushes $PC + 2$ onto the 5-level hardware LIFO return stack, then branches: $PC \leftarrow$ Address.
- **Subroutine Return (В/О – *Возврат / Отмена*):** 1-step instruction: Opcode `52`. Pops the top address from the hardware return stack and transfers execution there.
- **Root Level Behavior:** If executed outside a subroutine (return stack empty), `В/О` resets the program counter to zero ($PC \leftarrow 00$).

4.3 Inverted Conditional Branch Mechanics

Architectural Quirk of the B3-34 / MK-56 Family: Inverted Branching

In contrast to standard computing architectures (such as x86 or ARM) where a jump occurs when a condition is met:

The Elektronika MK-56 BRANCHES ONLY IF THE TEST CONDITION IS FALSE (*условие НЕ выполняется*).

If the test condition evaluates to **TRUE**, the processor **ignores the branch address** at $N+1$ and proceeds sequentially to the next instruction ($PC \leftarrow N+2$).

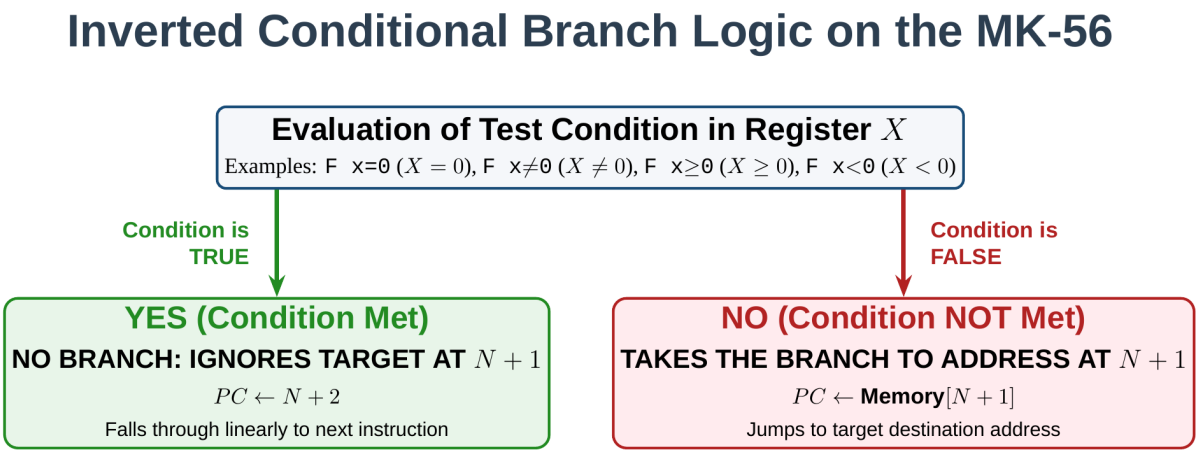


Figure 3: Inverted Conditional Branch Decision Flowchart on the Elektronika MK-56.

4.3.1 Formal Conditional Branch Specifications

1. `F x=0` (**Opcode `5E`**):
   - If $X \neq 0$ (False): **Branches** to address at $N + 1$.
   - If $X = 0$ (True): **Does not branch**; advances to $N + 2$.
2. `F x≠0` (**Opcode `57`**):
   - If $X = 0$ (False): **Branches** to address at $N + 1$.
   - If $X \neq 0$ (True): **Does not branch**; advances to $N + 2$.
3. `F x≥0` (**Opcode `59`**):
   - If $X < 0$ (False): **Branches** to address at $N + 1$.
   - If $X \geq 0$ (True): **Does not branch**; advances to $N + 2$.
4. `F x<0` (**Opcode `5C`**):
   - If $X \geq 0$ (False): **Branches** to address at $N + 1$.
   - If $X < 0$ (True): **Does not branch**; advances to $N + 2$.

4.3.2 Compiler Code Generation Pattern for Conditional Constructs

To compile a high-level construct such as:

```
if (X == 0) { ThenBlock } else { ElseBlock }
```

```
; At step N:
F x=0           ; Opcode 5E: Branch to else if X != 0
label_else      ; Address of Else block (step N+1)
; ThenBlock begins here (executes linearly if X == 0)
...
БП              ; Opcode 51: Unconditional jump over Else block
label_end       ; Exit address
label_else:
; ElseBlock begins here (reached if X != 0)
...
label_end:
```

4.4 Hardware Loop Counters: `F L0`, `F L1`, `F L2`, `F L3`

Specialized 2-step loop control instructions providing automated decrement and termination tests:

- **Loop Opcodes:** `F L0` → `5Г` ($RG0$); `F L1` → `5L` ($RG1$); `F L2` → `58` ($RG2$); `F L3` → `5-` ($RG3$).
- **Execution Mechanics:**
  1. Decrements the integer unit of the designated register:
  $$RGk \leftarrow RGk - 1 \qquad (k \in \{0, 1, 2, 3\})$$
  2. Evaluates the resulting value:
     - **If** $RGk \neq 0$**:** The loop continues; jumps to address at $N + 1$ ($PC \leftarrow$ Target).
     - **If** $RGk = 0$**:** The loop terminates; continues linearly to step $N + 2$ ($PC \leftarrow N + 2$).

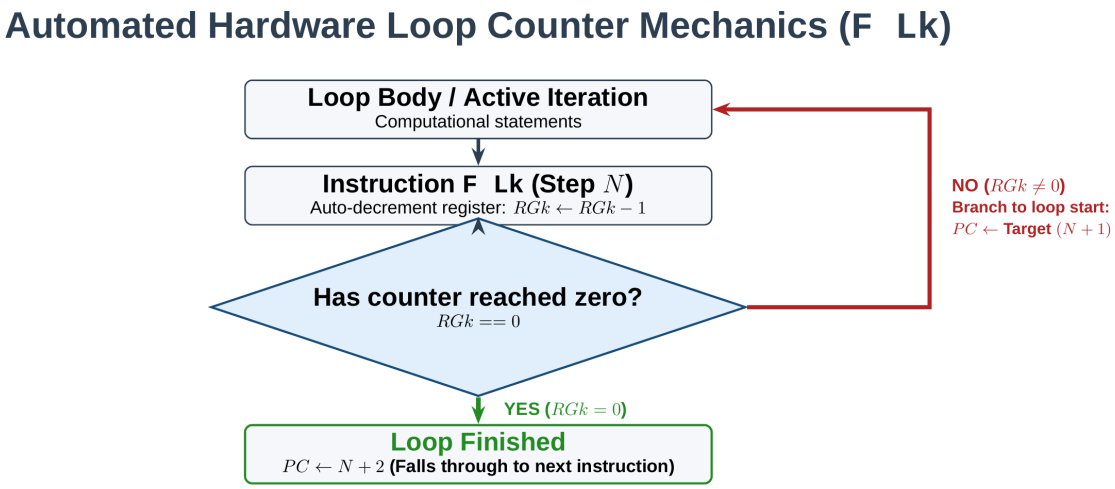


Figure 4: Control Cycle Flowchart for Hardware Automated Loop Counters (F Lk).

4.5 Indirect Branching and Dynamic Dispatch

The MK-56 supports indirect unconditional branches, indirect subroutine calls, and indirect conditional tests where the target address is **dynamically loaded from a data register**. These instructions occupy **a single program byte**:

- **Indirect Unconditional Jump:** K БП [R] (Opcodes 80 to 8Г). Modifies R (if $R \leq 6$) and branches to the integer address in R.
- **Indirect Subroutine Call:** K ПП [R] (Opcodes -0 to -Г). Pushes return address ($PC+1$) to return stack, modifies R, and branches.
- **Indirect Conditional Branches:** K x=0, K x≠0, K x≥0, K x<0. Evaluates X; if condition is false, modifies R and branches to address in R. If true, advances to $PC + 1$ without modifying R.

5 Architectural Specification Summary for Compiler Engineers

Architectural Parameter	Technical Specification
Execution Model	Pure Reverse Polish Notation (RPN) with top-of-stack accumulator (X)
Operational Stack Depth	4 hardware levels: X, Y, Z, T
Backup Register	1 automatic shadow register: X_1 (*Last X*)
Subroutine Call Stack Depth	5 dedicated hardware LIFO levels
Code Memory (RAM)	98 addressable program steps (`00` to `97`)
Data Registers	14 floating-point registers ($RG0..RG9, RGa..RGd$)
Opcode Length	1 byte per instruction; branch address in subsequent byte
Conditional Branch Logic	**Inverted:** branches when condition is FALSE; falls through when TRUE
Auto-decrementing Registers	$RG0, RG1, RG2, RG3$ (decremented by 1 when accessed via `K`)
Auto-incrementing Registers	$RG4, RG5, RG6$ (incremented by 1 when accessed via `K`)
Hardware Loop Counters	4 dedicated instructions: `F L0`, `F L1`, `F L2`, `F L3`
VFD Display Glyphs	Standard `0..9`; `A`→`-`, `B`→`L`, `C`→`C`, `D`→`Г`, `E`→`E`, `F`→*blank*

5.1 Practical IR Code Generation Guidelines

When constructing a compiler code generator targeting MK-56 bytecode:

1. **Static Stack Depth Tracking:** The compiler must compute stack depth at each IR node. Because the operational stack is limited to 4 levels, expressions requiring more than 4 concurrent intermediate values must spill temporary registers (typically $RG7..RGd$) using `x`→`П` and restore them via `П`→`x`.

2. **Conditional Inversion Mapping:** When emitting conditional jumps:

$$\text{Emit } \texttt{F x} = \texttt{0} \implies \text{Branch to failure label if } X \neq 0$$

$$\text{Emit } \texttt{F x} \neq \texttt{0} \implies \text{Branch to failure label if } X = 0$$

$$\text{Emit } \texttt{F x} \geq \texttt{0} \implies \text{Branch to failure label if } X < 0$$

$$\text{Emit } \texttt{F x} < \texttt{0} \implies \text{Branch to failure label if } X \geq 0$$

3. **Loop Optimization:** For bounded loops with iteration counts known at runtime in the range $1..99$, allocate the loop counter into one of $RG0..RG3$ and close the loop using `F Lk`, eliminating separate decrement and branch instructions.